

Với điều kiện ghi nguồn đầy đủ, Google cho phép tái tạo các bảng và hình trong bài báo này chỉ để sử dụng trong các tác phẩm báo chí hoặc học thuật.
Bản dịch tiếng Việt phục vụ mục đích học thuật và tham khảo.

Cơ chế chú ý là tất cả những gì bạn cần

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez*[†]
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaiser@google.com

Illia Polosukhin*[‡]
illia.polosukhin@gmail.com

Tóm tắt

Các mô hình chuyển đổi chuỗi chủ đạo hiện nay dựa trên các mạng nơ-ron hồi quy hoặc tích chập phức tạp, bao gồm một encoder và một decoder. Những mô hình có hiệu năng tốt nhất cũng kết nối encoder và decoder thông qua một cơ chế chú ý. Chúng tôi đề xuất một kiến trúc mạng mới, đơn giản, gọi là Transformer, chỉ dựa trên các cơ chế chú ý và loại bỏ hoàn toàn hồi quy cũng như tích chập. Các thí nghiệm trên hai tác vụ dịch máy cho thấy những mô hình này vượt trội về chất lượng, đồng thời có khả năng song song hóa cao hơn và cần ít thời gian huấn luyện hơn đáng kể. Mô hình của chúng tôi đạt 28.4 BLEU trên tác vụ dịch WMT 2014 English-to-German, cải thiện hơn 2 BLEU so với các kết quả tốt nhất hiện có, kể cả ensemble. Trên tác vụ dịch WMT 2014 English-to-French, mô hình của chúng tôi thiết lập điểm BLEU hiện đại nhất mới cho mô hình đơn là 41.8 sau khi huấn luyện 3,5 ngày trên tám GPU, chỉ bằng một phần nhỏ chi phí huấn luyện của các mô hình tốt nhất trong tài liệu. Chúng tôi cho thấy Transformer khá quát hóa tốt sang các tác vụ khác bằng cách áp dụng thành công nó cho phân tích cú pháp cấu trúc câu tiếng Anh với cả dữ liệu huấn luyện lớn và hạn chế.

*Đóng góp ngang nhau. Thứ tự liệt kê là ngẫu nhiên. Jakob đề xuất thay thế RNN bằng tự chú ý và khởi xướng nỗ lực đánh giá ý tưởng này. Ashish, cùng với Illia, đã thiết kế và triển khai các mô hình Transformer đầu tiên và tham gia then chốt vào mọi khía cạnh của công trình này. Noam đề xuất chú ý tích vô hướng có tỷ lệ, chú ý đa đầu và biểu diễn vị trí không tham số, đồng thời trở thành người còn lại tham gia gần như mọi chi tiết. Niki thiết kế, triển khai, tinh chỉnh và đánh giá vô số biến thể mô hình trong codebase gốc của chúng tôi và tensor2tensor. Llion cũng thử nghiệm các biến thể mô hình mới, chịu trách nhiệm về codebase ban đầu của chúng tôi, suy luận hiệu quả và trực quan hóa. Lukasz và Aidan đã dành nhiều ngày dài để thiết kế các phần khác nhau và triển khai tensor2tensor, thay thế codebase trước đó của chúng tôi, cải thiện mạnh mẽ kết quả và tăng tốc đáng kể nghiên cứu của chúng tôi.

[†]Công việc được thực hiện khi còn ở Google Brain.

[‡]Công việc được thực hiện khi còn ở Google Research.

1 Giới thiệu

Các mạng nơ-ron hồi quy, đặc biệt là bộ nhớ dài-ngắn hạn (long short-term memory, LSTM) [13] và mạng nơ-ron hồi quy có cổng (gated recurrent neural networks, GRU) [7], từ lâu đã được xem là các phương pháp hiện đại nhất cho các bài toán mô hình hóa chuỗi và chuyển đổi chuỗi như mô hình hóa ngôn ngữ và dịch máy [35, 2, 5]. Nhiều nỗ lực sau đó tiếp tục mở rộng giới hạn của các mô hình ngôn ngữ hồi quy và kiến trúc encoder-decoder [38, 24, 15].

Các mô hình hồi quy thường phân rã phép tính theo các vị trí ký hiệu trong chuỗi đầu vào và đầu ra. Bằng cách căn chỉnh các vị trí với các bước theo thời gian tính toán, chúng sinh ra một chuỗi trạng thái ẩn h_t như một hàm của trạng thái ẩn trước đó h_{t-1} và đầu vào tại vị trí t . Tính chất tuần tự vốn có này cản trở việc song song hóa bên trong từng ví dụ huấn luyện; điều này trở nên đặc biệt quan trọng khi độ dài chuỗi tăng lên, vì ràng buộc bộ nhớ giới hạn khả năng gom batch trên nhiều ví dụ. Các nghiên cứu gần đây đã đạt được những cải thiện đáng kể về hiệu quả tính toán thông qua các thủ thuật phân rã [21] và tính toán có điều kiện [32], trong trường hợp thứ hai còn cải thiện cả hiệu năng mô hình. Tuy nhiên, ràng buộc nền tảng của tính toán tuần tự vẫn còn tồn tại.

Các cơ chế chú ý đã trở thành một thành phần không thể thiếu của các mô hình mô hình hóa chuỗi và chuyển đổi chuỗi có sức thuyết phục trong nhiều tác vụ khác nhau, cho phép mô hình hóa các phụ thuộc mà không phụ thuộc vào khoảng cách của chúng trong chuỗi đầu vào hoặc đầu ra [2, 19]. Tuy nhiên, ngoại trừ một vài trường hợp [27], các cơ chế chú ý như vậy vẫn được dùng kết hợp với mạng hồi quy.

Trong công trình này, chúng tôi đề xuất Transformer, một kiến trúc mô hình loại bỏ hồi quy và thay vào đó dựa hoàn toàn vào cơ chế chú ý để nắm bắt các phụ thuộc toàn cục giữa đầu vào và đầu ra. Transformer cho phép song song hóa nhiều hơn đáng kể và có thể đạt chất lượng dịch mới ở mức hiện đại nhất sau khi được huấn luyện chỉ trong mười hai giờ trên tám GPU P100.

2 Bối cảnh

Mục tiêu giảm tính toán tuần tự cũng là nền tảng của Extended Neural GPU [16], ByteNet [18] và ConvS2S [9]; tất cả đều dùng mạng nơ-ron tích chập làm khối xây dựng cơ bản và tính các biểu diễn ẩn song song cho mọi vị trí đầu vào và đầu ra. Trong các mô hình này, số phép toán cần thiết để liên hệ tín hiệu từ hai vị trí đầu vào hoặc đầu ra bất kỳ tăng theo khoảng cách giữa các vị trí: tuyến tính đối với ConvS2S và lôgarit đối với ByteNet. Điều này làm cho việc học các phụ thuộc giữa những vị trí xa nhau trở nên khó hơn [12]. Trong Transformer, điều này được giảm xuống còn một số lượng phép toán hằng số, mặc dù phải đánh đổi bằng độ phân giải hiệu dụng thấp hơn do việc lấy trung bình các vị trí được gán trọng số chú ý; chúng tôi khắc phục hiệu ứng này bằng Multi-Head Attention như mô tả trong phần 3.2.

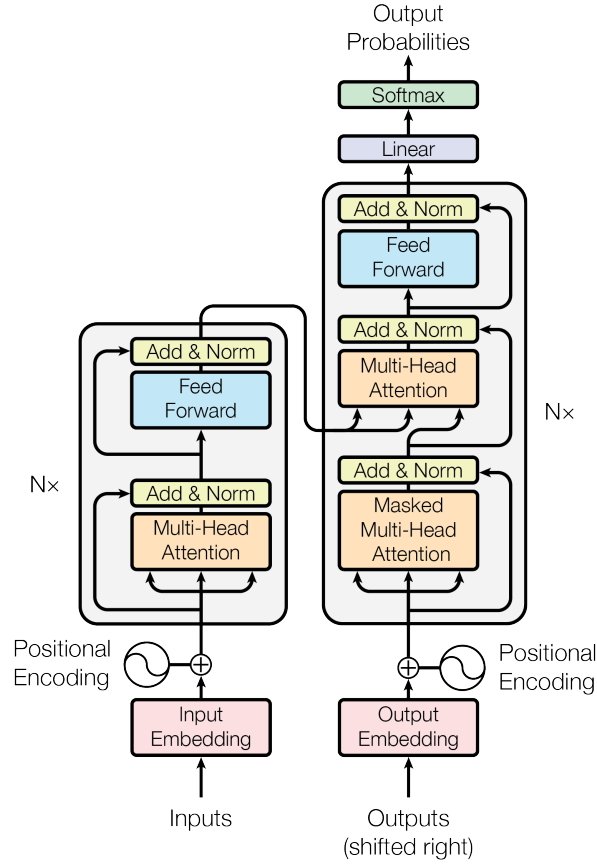
Tự chú ý (self-attention), đôi khi được gọi là chú ý nội tại (intra-attention), là một cơ chế chú ý liên hệ các vị trí khác nhau của cùng một chuỗi để tính biểu diễn của chuỗi đó. Tự chú ý đã được sử dụng thành công trong nhiều tác vụ, bao gồm đọc hiểu, tóm tắt trừu tượng, suy luận văn bản và học biểu diễn câu độc lập với tác vụ [4, 27, 28, 22].

Các mạng bộ nhớ đầu-cuối (end-to-end memory networks) dựa trên cơ chế chú ý hồi quy thay vì hồi quy căn theo chuỗi, và đã được chứng minh là hoạt động tốt trên các tác vụ hỏi đáp ngôn ngữ đơn giản và mô hình hóa ngôn ngữ [34].

Theo hiểu biết tốt nhất của chúng tôi, Transformer là mô hình chuyển đổi đầu tiên dựa hoàn toàn vào tự chú ý để tính biểu diễn của đầu vào và đầu ra mà không dùng RNN căn theo chuỗi hay tích chập. Trong các phần tiếp theo, chúng tôi sẽ mô tả Transformer, nêu động lực cho việc dùng tự chú ý và thảo luận các ưu điểm của nó so với các mô hình như [17, 18] và [9].

3 Kiến trúc mô hình

Hầu hết các mô hình chuyển đổi chuỗi bằng mạng nơ-ron có tính cạnh tranh đều có cấu trúc encoder-decoder [5, 2, 35]. Ở đây, bộ mã hóa ánh xạ một chuỗi đầu vào gồm các biểu diễn ký hiệu $(x_1, \dots, x_n)$ thành một chuỗi biểu diễn liên tục $\mathbf{z} = (z_1, \dots, z_n)$. Với $\mathbf{z}$, bộ giải mã sau đó sinh ra chuỗi đầu ra $(y_1, \dots, y_m)$ gồm các ký hiệu, mỗi lần một phần tử. Ở mỗi bước, mô hình có tính tự hồi quy [10], sử dụng các ký hiệu đã sinh trước đó như đầu vào bổ sung khi sinh ký hiệu tiếp theo.



Hình 1: Transformer - kiến trúc mô hình.

Transformer tuân theo kiến trúc tổng quát này bằng cách dùng các lớp tự chú ý xếp chồng và các lớp kết nối đầy đủ theo từng vị trí cho cả encoder và decoder, lần lượt được minh họa ở nửa trái và nửa phải của Hình 1.

3.1 Các ngăn xếp Encoder và Decoder

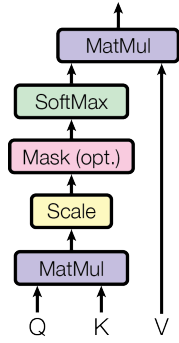
Encoder: Encoder gồm một ngăn xếp $N = 6$ lớp giống hệt nhau. Mỗi lớp có hai lớp con. Lớp con thứ nhất là một cơ chế tự chú ý đa đầu, và lớp con thứ hai là một mạng truyền thẳng kết nối đầy đủ đơn giản theo từng vị trí. Chúng tôi dùng kết nối tắt (residual connection) [11] quanh mỗi lớp con trong hai lớp con, theo sau là chuẩn hóa lớp [1]. Cụ thể, đầu ra của mỗi lớp con là $\text{LayerNorm}(x + \text{Sublayer}(x))$, trong đó $\text{Sublayer}(x)$ là hàm do chính lớp con đó thực hiện. Để hỗ trợ các kết nối tắt này, tất cả lớp con trong mô hình, cũng như các lớp embedding, đều tạo ra đầu ra có số chiều $d_{\text{model}} = 512$.

Decoder: Decoder cũng gồm một ngăn xếp $N = 6$ lớp giống hệt nhau. Ngoài hai lớp con trong mỗi lớp encoder, decoder chèn thêm một lớp con thứ ba, thực hiện chú ý đa đầu trên đầu ra của ngăn xếp encoder. Tương tự encoder, chúng tôi dùng các kết nối tắt quanh mỗi lớp con, theo sau là chuẩn hóa lớp. Chúng tôi cũng sửa lớp con tự chú ý trong ngăn xếp decoder để ngăn các vị trí chú ý tới những vị trí phía sau. Việc che mặt nạ này, kết hợp với thực tế rằng các embedding đầu ra được dịch đi một vị trí, bảo đảm rằng các dự đoán cho vị trí i chỉ có thể phụ thuộc vào các đầu ra đã biết tại những vị trí nhỏ hơn i .

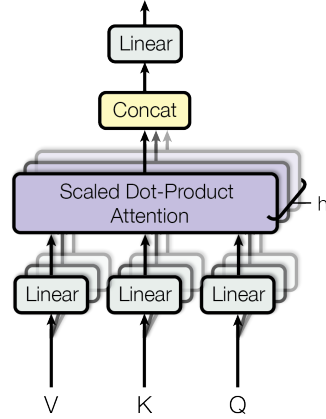
3.2 Chú ý

Một hàm chú ý có thể được mô tả như phép ánh xạ một truy vấn và một tập các cặp khóa-giá trị thành một đầu ra, trong đó truy vấn, khóa, giá trị và đầu ra đều là các vector. Đầu ra được tính như

Chú ý tích vô hướng có tỷ lệ



Chú ý đa đầu



Hình 2: (trái) Chú ý tích vô hướng có tỷ lệ. (phải) Chú ý đa đầu gồm nhiều lớp chú ý chạy song song.

tổng có trọng số của các giá trị, trong đó trọng số gán cho mỗi giá trị được tính bằng một hàm tương thích giữa truy vấn và khóa tương ứng.

3.2.1 Chú ý tích vô hướng có tỷ lệ

Chúng tôi gọi cơ chế chú ý cụ thể của mình là “Scaled Dot-Product Attention” (Hình 2). Đầu vào gồm các truy vấn và khóa có số chiều d_k , cùng các giá trị có số chiều d_v . Chúng tôi tính các tích vô hướng của truy vấn với tất cả khóa, chia mỗi tích cho $\sqrt{d_k}$, rồi áp dụng hàm softmax để thu được các trọng số trên các giá trị.

Trong thực tế, chúng tôi tính hàm chú ý trên một tập truy vấn đồng thời, được đóng gói cùng nhau thành một ma trận Q . Các khóa và giá trị cũng được đóng gói cùng nhau thành các ma trận K và V . Chúng tôi tính ma trận đầu ra như sau:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1)$$

Hai hàm chú ý được dùng phổ biến nhất là chú ý cộng (additive attention) [2] và chú ý tích vô hướng, tức chú ý nhân (dot-product/multiplicative attention). Chú ý tích vô hướng giống hệt thuật toán của chúng tôi, ngoại trừ hệ số tỷ lệ $\frac{1}{\sqrt{d_k}}$. Chú ý cộng tính hàm tương thích bằng một mạng truyền thẳng có một lớp ẩn. Mặc dù hai cơ chế có độ phức tạp lý thuyết tương tự nhau, chú ý tích vô hướng nhanh hơn nhiều và hiệu quả không gian hơn trong thực tế, vì có thể được triển khai bằng mã nhân ma trận đã tối ưu hóa cao.

Với các giá trị nhỏ của d_k , hai cơ chế hoạt động tương tự nhau; tuy nhiên, với các giá trị lớn hơn của d_k , chú ý cộng vượt trội hơn chú ý tích vô hướng không tỷ lệ [3]. Chúng tôi nghi ngờ rằng với các giá trị lớn của d_k , các tích vô hướng có độ lớn tăng cao, đẩy hàm softmax vào những vùng có gradient cực nhỏ⁴. Để chống lại hiệu ứng này, chúng tôi tỷ lệ hóa các tích vô hướng bằng $\frac{1}{\sqrt{d_k}}$.

3.2.2 Chú ý đa đầu

Thay vì thực hiện một hàm chú ý duy nhất với các khóa, giá trị và truy vấn có số chiều d_{model} , chúng tôi nhận thấy việc chiếu tuyến tính các truy vấn, khóa và giá trị h lần bằng các phép chiếu tuyến tính khác nhau, được học, lần lượt sang các số chiều d_k , d_k và d_v là có lợi. Trên mỗi phiên bản đã được

⁴Để minh họa vì sao các tích vô hướng trở nên lớn, giả sử các thành phần của q và k là các biến ngẫu nhiên độc lập với trung bình 0 và phương sai 1. Khi đó tích vô hướng của chúng, $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$, có trung bình 0 và phương sai d_k .

chiều của truy vấn, khóa và giá trị, chúng tôi thực hiện hàm chú ý song song, tạo ra các giá trị đầu ra có số chiều d_v . Các giá trị này được nối lại rồi một lần nữa được chiếu, tạo ra các giá trị cuối cùng như minh họa trong Hình 2.

Chú ý đa đầu cho phép mô hình cùng lúc chú ý tới thông tin từ các không gian con biểu diễn khác nhau tại các vị trí khác nhau. Với chỉ một đầu chú ý, việc lấy trung bình sẽ hạn chế điều này.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

$$\text{where head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Trong đó các phép chiếu là những ma trận tham số $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ và $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$.

Trong công trình này, chúng tôi dùng $h = 8$ lớp chú ý song song, hay các đầu chú ý. Với mỗi đầu, chúng tôi dùng $d_k = d_v = d_{\text{model}}/h = 64$. Nhờ số chiều của mỗi đầu nhỏ hơn, tổng chi phí tính toán tương tự như chú ý một đầu với đầy đủ số chiều.

3.2.3 Các ứng dụng của chú ý trong mô hình của chúng tôi

Transformer dùng chú ý đa đầu theo ba cách khác nhau:

- Trong các lớp “chú ý encoder-decoder”, các truy vấn đến từ lớp decoder trước đó, còn các khóa và giá trị bộ nhớ đến từ đầu ra của encoder. Điều này cho phép mọi vị trí trong decoder chú ý tới tất cả vị trí trong chuỗi đầu vào. Cơ chế này bắt chước các cơ chế chú ý encoder-decoder điển hình trong các mô hình sequence-to-sequence như [38, 2, 9].
- Encoder chứa các lớp tự chú ý. Trong một lớp tự chú ý, tất cả khóa, giá trị và truy vấn đều đến từ cùng một nơi, trong trường hợp này là đầu ra của lớp trước đó trong encoder. Mỗi vị trí trong encoder có thể chú ý tới tất cả vị trí trong lớp trước đó của encoder.
- Tương tự, các lớp tự chú ý trong decoder cho phép mỗi vị trí trong decoder chú ý tới tất cả vị trí trong decoder cho tới và bao gồm cả vị trí đó. Chúng ta cần ngăn luồng thông tin từ bên phải sang bên trái trong decoder để bảo toàn tính tự hồi quy. Chúng tôi triển khai điều này bên trong chú ý tích vô hướng có tỷ lệ bằng cách che mặt nạ, tức đặt thành $-\infty$, mọi giá trị trong đầu vào của softmax tương ứng với các kết nối không hợp lệ. Xem Hình 2.

3.3 Mạng truyền thẳng theo từng vị trí

Ngoài các lớp con chú ý, mỗi lớp trong encoder và decoder của chúng tôi chứa một mạng truyền thẳng kết nối đầy đủ, được áp dụng riêng rẽ và giống hệt nhau cho từng vị trí. Mạng này gồm hai phép biến đổi tuyến tính với một kích hoạt ReLU ở giữa.

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (2)$$

Mặc dù các phép biến đổi tuyến tính là giống nhau trên các vị trí khác nhau, chúng dùng các tham số khác nhau giữa các lớp. Một cách mô tả khác là xem chúng như hai phép tích chập với kích thước kernel bằng 1. Số chiều đầu vào và đầu ra là $d_{\text{model}} = 512$, còn lớp bên trong có số chiều $d_{ff} = 2048$.

3.4 Embedding và Softmax

Tương tự các mô hình chuyển đổi chuỗi khác, chúng tôi dùng các embedding được học để chuyển các token đầu vào và token đầu ra thành các vector có số chiều d_{model} . Chúng tôi cũng dùng phép biến đổi tuyến tính được học thông thường và hàm softmax để chuyển đầu ra của decoder thành xác suất dự đoán token tiếp theo. Trong mô hình của chúng tôi, cùng một ma trận trọng số được chia sẻ giữa hai lớp embedding và phép biến đổi tuyến tính trước softmax, tương tự [30]. Trong các lớp embedding, chúng tôi nhân các trọng số đó với $\sqrt{d_{\text{model}}}$.

Bảng 1: Độ dài đường đi tối đa, độ phức tạp theo từng lớp và số phép toán tuần tự tối thiểu đối với các loại lớp khác nhau. n là độ dài chuỗi, d là số chiều biểu diễn, k là kích thước kernel của tích chập và r là kích thước lân cận trong tự chú ý bị giới hạn.

Loại lớp	Độ phức tạp mỗi lớp	Phép toán tuần tự	Độ dài đường đi tối đa
Tự chú ý	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Hồi quy	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Tích chập	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Tự chú ý (giới hạn)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

3.5 Mã hóa vị trí

Vì mô hình của chúng tôi không chứa hồi quy và không chứa tích chập, để mô hình có thể sử dụng thứ tự của chuỗi, chúng tôi phải đưa vào một số thông tin về vị trí tương đối hoặc tuyệt đối của các token trong chuỗi. Vì mục đích này, chúng tôi cộng “mã hóa vị trí” vào các embedding đầu vào ở đáy của các ngăn xếp encoder và decoder. Các mã hóa vị trí có cùng số chiều d_{model} như các embedding, nhờ đó hai thành phần có thể được cộng với nhau. Có nhiều lựa chọn cho mã hóa vị trí, cả dạng học được và dạng cố định [9].

Trong công trình này, chúng tôi dùng các hàm sin và cos với những tần số khác nhau:

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$

trong đó pos là vị trí và i là số chiều. Nói cách khác, mỗi chiều của mã hóa vị trí tương ứng với một hình sin. Các bước sóng tạo thành một cặp số nhân từ 2π tới $10000 \cdot 2\pi$. Chúng tôi chọn hàm này vì giả thuyết rằng nó sẽ cho phép mô hình dễ dàng học cách chú ý theo vị trí tương đối, do với mọi độ lệch cố định k , PE_{pos+k} có thể được biểu diễn như một hàm tuyến tính của PE_{pos} .

Chúng tôi cũng thử nghiệm dùng các embedding vị trí được học [9] thay thế và nhận thấy hai phiên bản cho kết quả gần như giống hệt nhau (xem hàng (E) của Bảng 3). Chúng tôi chọn phiên bản hình sin vì nó có thể cho phép mô hình ngoại suy sang các độ dài chuỗi lớn hơn những độ dài đã gặp trong huấn luyện.

4 Vì sao dùng tự chú ý

Trong phần này, chúng tôi so sánh nhiều khía cạnh của các lớp tự chú ý với các lớp hồi quy và tích chập thường được dùng để ánh xạ một chuỗi biểu diễn ký hiệu có độ dài biến đổi $(x_1, \dots, x_n)$ sang một chuỗi cùng độ dài $(z_1, \dots, z_n)$, với $x_i, z_i \in \mathbb{R}^d$, chẳng hạn như một lớp ẩn trong encoder hoặc decoder chuyển đổi chuỗi điển hình. Để nêu động lực cho việc dùng tự chú ý, chúng tôi xét ba tiêu chí mong muốn.

Thứ nhất là tổng độ phức tạp tính toán trên mỗi lớp. Thứ hai là lượng tính toán có thể được song song hóa, được đo bằng số phép toán tuần tự tối thiểu cần thực hiện.

Thứ ba là độ dài đường đi giữa các phụ thuộc tầm xa trong mạng. Học các phụ thuộc tầm xa là một thách thức then chốt trong nhiều tác vụ chuyển đổi chuỗi. Một yếu tố quan trọng ảnh hưởng tới khả năng học các phụ thuộc như vậy là độ dài của các đường đi mà tín hiệu truyền xuôi và truyền ngược phải đi qua trong mạng. Các đường đi giữa bất kỳ tổ hợp vị trí nào trong chuỗi đầu vào và đầu ra càng ngắn thì việc học các phụ thuộc tầm xa càng dễ hơn [12]. Vì vậy, chúng tôi cũng so sánh độ dài đường đi tối đa giữa bất kỳ hai vị trí đầu vào và đầu ra nào trong các mạng được cấu thành từ những loại lớp khác nhau.

Như đã nêu trong Bảng 1, một lớp tự chú ý kết nối mọi vị trí bằng một số lượng phép toán thực hiện tuần tự không đổi, trong khi một lớp hồi quy cần $O(n)$ phép toán tuần tự. Về độ phức tạp tính toán, các lớp tự chú ý nhanh hơn các lớp hồi quy khi độ dài chuỗi n nhỏ hơn số chiều biểu diễn d , điều thường xảy ra với các biểu diễn câu được sử dụng bởi những mô hình dịch máy hiện đại nhất, chẳng

hạn các biểu diễn word-piece [38] và byte-pair [31]. Để cải thiện hiệu năng tính toán cho các tác vụ liên quan đến chuỗi rất dài, tự chú ý có thể được giới hạn để chỉ xét một lân cận kích thước r trong chuỗi đầu vào, lấy tâm tại vị trí đầu ra tương ứng. Điều này sẽ làm tăng độ dài đường đi tối đa lên $O(n/r)$. Chúng tôi dự định khảo sát hướng tiếp cận này sâu hơn trong tương lai.

Một lớp tích chập đơn lẻ với độ rộng kernel $k < n$ không kết nối mọi cặp vị trí đầu vào và đầu ra. Để làm được điều đó cần một ngăn xếp gồm $O(n/k)$ lớp tích chập trong trường hợp kernel liên tiếp, hoặc $O(\log_k(n))$ trong trường hợp tích chập giãn [18], làm tăng độ dài của các đường đi dài nhất giữa hai vị trí bất kỳ trong mạng. Các lớp tích chập nhìn chung đắt hơn các lớp hồi quy một hệ số k . Tuy nhiên, tích chập tách được (separable convolutions) [6] làm giảm độ phức tạp đáng kể xuống còn $O(k \cdot n \cdot d + n \cdot d^2)$. Ngay cả khi $k = n$, độ phức tạp của một phép tích chập tách được vẫn bằng tổ hợp của một lớp tự chú ý và một lớp truyền thẳng theo từng vị trí, chính là cách tiếp cận mà chúng tôi dùng trong mô hình.

Như một lợi ích phụ, tự chú ý có thể tạo ra các mô hình dễ diễn giải hơn. Chúng tôi khảo sát các phân bố chú ý từ mô hình của mình, đồng thời trình bày và thảo luận các ví dụ trong phụ lục. Không chỉ từng đầu chú ý riêng lẻ học rõ ràng để thực hiện các tác vụ khác nhau, nhiều đầu còn dường như biểu hiện hành vi liên quan đến cấu trúc cú pháp và ngữ nghĩa của câu.

5 Huấn luyện

Phần này mô tả chế độ huấn luyện cho các mô hình của chúng tôi.

5.1 Dữ liệu huấn luyện và gom batch

Chúng tôi huấn luyện trên bộ dữ liệu WMT 2014 English-German chuẩn, gồm khoảng 4,5 triệu cặp câu. Các câu được mã hóa bằng mã hóa byte-pair [3], với bộ từ vựng nguồn-đích dùng chung gồm khoảng 37000 token. Đối với English-French, chúng tôi dùng bộ dữ liệu WMT 2014 English-French lớn hơn đáng kể, gồm 36 triệu câu, và tách token thành một bộ từ vựng word-piece gồm 32000 mục [38]. Các cặp câu được gom batch theo độ dài chuỗi xấp xỉ. Mỗi batch huấn luyện chứa một tập cặp câu với khoảng 25000 token nguồn và 25000 token đích.

5.2 Phân cứng và lịch huấn luyện

Chúng tôi huấn luyện các mô hình trên một máy có 8 GPU NVIDIA P100. Với các mô hình cơ sở dùng những siêu tham số được mô tả xuyên suốt bài báo, mỗi bước huấn luyện mất khoảng 0,4 giây. Chúng tôi huấn luyện các mô hình cơ sở trong tổng cộng 100000 bước, tức 12 giờ. Với các mô hình lớn, được mô tả ở dòng cuối của Bảng 3, thời gian mỗi bước là 1,0 giây. Các mô hình lớn được huấn luyện trong 300000 bước, tức 3,5 ngày.

5.3 Bộ tối ưu

Chúng tôi dùng bộ tối ưu Adam [20] với $\beta_1 = 0.9$, $\beta_2 = 0.98$ và $\epsilon = 10^{-9}$. Chúng tôi thay đổi learning rate trong quá trình huấn luyện theo công thức:

$$lr_{rate} = d_{model}^{-0.5} \cdot \min(step_num^{-0.5}, step_num \cdot warmup_steps^{-1.5}) \quad (3)$$

Điều này tương ứng với việc tăng learning rate tuyến tính trong $warmup_steps$ bước huấn luyện đầu tiên, rồi sau đó giảm tỷ lệ thuận với nghịch đảo căn bậc hai của số bước. Chúng tôi dùng $warmup_steps = 4000$.

5.4 Chính quy hóa

Chúng tôi dùng ba loại chính quy hóa trong huấn luyện:

Dropout phần dư Chúng tôi áp dụng dropout [33] lên đầu ra của mỗi lớp con, trước khi nó được cộng vào đầu vào của lớp con và được chuẩn hóa. Ngoài ra, chúng tôi áp dụng dropout lên tổng của các embedding và mã hóa vị trí trong cả ngăn xếp encoder lẫn decoder. Với mô hình cơ sở, chúng tôi dùng tỷ lệ $P_{drop} = 0.1$.

Bảng 2: Transformer đạt điểm BLEU tốt hơn các mô hình hiện đại nhất trước đó trên các bộ kiểm thử English-to-German và English-to-French newstest2014, với chi phí huấn luyện chỉ bằng một phần nhỏ.

Mô hình	BLEU		Chi phí huấn luyện (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (mô hình cơ sở)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (lớn)	28.4	41.8	$2.3 \cdot 10^{19}$	

Làm mượt nhân Trong huấn luyện, chúng tôi dùng làm mượt nhân với giá trị $\epsilon_{ls} = 0.1$ [36]. Điều này làm xấu perplexity vì mô hình học cách kém chắc chắn hơn, nhưng cải thiện độ chính xác và điểm BLEU.

6 Kết quả

6.1 Dịch máy

Trên tác vụ dịch WMT 2014 English-to-German, mô hình transformer lớn, tức Transformer (lớn) trong Bảng 2, vượt trội hơn các mô hình tốt nhất đã được báo cáo trước đó, bao gồm cả ensemble, hơn 2.0 BLEU, qua đó thiết lập điểm BLEU mới ở mức hiện đại nhất là 28.4. Cấu hình của mô hình này được liệt kê ở dòng cuối của Bảng 3. Quá trình huấn luyện mất 3.5 ngày trên 8 GPU P100. Ngay cả mô hình cơ sở của chúng tôi cũng vượt qua tất cả mô hình và ensemble đã công bố trước đó, với chi phí huấn luyện chỉ bằng một phần nhỏ so với bất kỳ mô hình cạnh tranh nào.

Trên tác vụ dịch WMT 2014 English-to-French, mô hình lớn của chúng tôi đạt điểm BLEU 41.0, vượt qua tất cả các mô hình đơn đã công bố trước đó, với chi phí huấn luyện thấp hơn 1/4 so với mô hình hiện đại nhất trước đó. Mô hình Transformer (lớn) được huấn luyện cho English-to-French dùng tỷ lệ dropout $P_{drop} = 0.1$, thay vì 0.3.

Đối với các mô hình cơ sở, chúng tôi dùng một mô hình đơn thu được bằng cách lấy trung bình 5 checkpoint cuối cùng, được ghi lại ở các khoảng cách 10 phút. Đối với các mô hình lớn, chúng tôi lấy trung bình 20 checkpoint cuối cùng. Chúng tôi dùng beam search với kích thước beam bằng 4 và hệ số phạt độ dài $\alpha = 0.6$ [38]. Các siêu tham số này được chọn sau khi thử nghiệm trên tập phát triển. Trong suy luận, chúng tôi đặt độ dài đầu ra tối đa bằng độ dài đầu vào + 50, nhưng kết thúc sớm khi có thể [38].

Bảng 2 tóm tắt kết quả của chúng tôi và so sánh chất lượng dịch cũng như chi phí huấn luyện với các kiến trúc mô hình khác trong tài liệu. Chúng tôi ước tính số phép toán dấu phẩy động dùng để huấn luyện một mô hình bằng cách nhân thời gian huấn luyện, số GPU được dùng và một ước lượng về năng lực tính toán dấu phẩy động đơn chính xác duy trì của mỗi GPU⁵.

6.2 Các biến thể mô hình

Để đánh giá tầm quan trọng của các thành phần khác nhau trong Transformer, chúng tôi biến đổi mô hình cơ sở theo nhiều cách, đo sự thay đổi hiệu năng trên bài toán dịch English-to-German ở tập phát triển newstest2013. Chúng tôi dùng beam search như mô tả trong phần trước, nhưng không lấy trung bình checkpoint. Chúng tôi trình bày các kết quả này trong Bảng 3.

⁵Chúng tôi dùng các giá trị 2.8, 3.7, 6.0 và 9.5 TFLOPS lần lượt cho K80, K40, M40 và P100.

Bảng 3: Các biến thể của kiến trúc Transformer. Những giá trị không liệt kê giống hệt mô hình cơ sở. Tất cả chỉ số đều được đo trên tập phát triển dịch English-to-German, newstest2013. Các perplexity được liệt kê tính theo từng wordpiece, theo mã hóa byte-pair của chúng tôi, và không nên so sánh với perplexity tính theo từng từ.

	N	d_{model}	d_{ff}	h	d_k	d_v	P_{drop}	ϵ_{ls}	train bước	PPL (dev)	BLEU (dev)	tham số $\times 10^6$		
base	6	512	2048	8	64	64	0.1	0.1	100K	4.92	25.8	65		
(A)					1	512	512				5.29	24.9		
					4	128	128				5.00	25.5		
					16	32	32				4.91	25.8		
					32	16	16				5.01	25.4		
(B)					16					5.16	25.1	58		
					32					5.01	25.4	60		
(C)	2									6.11	23.7	36		
	4									5.19	25.3	50		
	8									4.88	25.5	80		
		256				32	32				5.75	24.5	28	
		1024				128	128				4.66	26.0	168	
			1024								5.12	25.4	53	
			4096								4.75	26.2	90	
(D)							0.0				5.77	24.6		
							0.2				4.95	25.5		
								0.0				4.67	25.3	
								0.2				5.47	25.7	
(E)	embedding vị trí thay cho các hình sin									4.92	25.7			
big	6	1024	4096	16				0.3	300K	4.33	26.4	213		

Trong các hàng (A) của Bảng 3, chúng tôi thay đổi số đầu chú ý và số chiều khóa cũng như giá trị của chú ý, trong khi giữ nguyên lượng tính toán như mô tả ở Phần 3.2.2. Mặc dù chú ý một đầu kém hơn cấu hình tốt nhất 0.9 BLEU, chất lượng cũng giảm khi có quá nhiều đầu.

Trong các hàng (B) của Bảng 3, chúng tôi quan sát thấy rằng việc giảm kích thước khóa chú ý d_k làm giảm chất lượng mô hình. Điều này cho thấy việc xác định độ tương thích là không dễ, và một hàm tương thích tinh vi hơn tích vô hướng có thể hữu ích. Chúng tôi tiếp tục quan sát ở các hàng (C) và (D) rằng, đúng như kỳ vọng, các mô hình lớn hơn tốt hơn, và dropout rất hữu ích trong việc tránh over-fitting. Ở hàng (E), chúng tôi thay mã hóa vị trí hình sin bằng các embedding vị trí được học [9] và quan sát được kết quả gần như giống hệt mô hình cơ sở.

6.3 Phân tích cú pháp cấu trúc câu tiếng Anh

Để đánh giá liệu Transformer có thể khái quát hóa sang các tác vụ khác hay không, chúng tôi thực hiện các thí nghiệm trên phân tích cú pháp cấu trúc câu tiếng Anh. Tác vụ này đặt ra các thách thức riêng: đầu ra chịu các ràng buộc cấu trúc mạnh và dài hơn đầu vào đáng kể. Hơn nữa, các mô hình RNN sequence-to-sequence chưa thể đạt kết quả hiện đại nhất trong các chế độ dữ liệu nhỏ [37].

Chúng tôi huấn luyện một transformer 4 lớp với $d_{\text{model}} = 1024$ trên phần Wall Street Journal (WSJ) của Penn Treebank [25], gồm khoảng 40K câu huấn luyện. Chúng tôi cũng huấn luyện nó trong thiết lập bán giám sát, dùng các tập ngữ liệu độ tin cậy cao lớn hơn và BerkleyParser với xấp xỉ 17M câu [37]. Chúng tôi dùng bộ từ vựng 16K token cho thiết lập chỉ dùng WSJ và bộ từ vựng 32K token cho thiết lập bán giám sát.

Chúng tôi chỉ thực hiện một số lượng nhỏ thí nghiệm để chọn dropout, gồm cả chú ý và residual (phần 5.4), learning rate và kích thước beam trên tập phát triển Section 22; tất cả tham số khác giữ nguyên so với mô hình dịch cơ sở English-to-German. Trong suy luận, chúng tôi tăng độ dài đầu ra tối đa lên độ dài đầu vào + 300. Chúng tôi dùng kích thước beam 21 và $\alpha = 0.3$ cho cả thiết lập chỉ dùng WSJ và bán giám sát.

Bảng 4: Transformer khái quát hóa tốt sang bài toán phân tích cú pháp cấu trúc câu tiếng Anh (kết quả trên Section 23 của WSJ)

Bộ phân tích	Huấn luyện	WSJ 23 F1
Vinyals & Kaiser et al. (2014) [37]	chỉ WSJ, phân biệt	88.3
Petrov et al. (2006) [29]	chỉ WSJ, phân biệt	90.4
Zhu et al. (2013) [40]	chỉ WSJ, phân biệt	90.4
Dyer et al. (2016) [8]	chỉ WSJ, phân biệt	91.7
Transformer (4 lớp)	chỉ WSJ, phân biệt	91.3
Zhu et al. (2013) [40]	bán giám sát	91.3
Huang & Harper (2009) [14]	bán giám sát	91.3
McClosky et al. (2006) [26]	bán giám sát	92.1
Vinyals & Kaiser et al. (2014) [37]	bán giám sát	92.1
Transformer (4 lớp)	bán giám sát	92.7
Luong et al. (2015) [23]	đa tác vụ	93.0
Dyer et al. (2016) [8]	sinh	93.3

Các kết quả của chúng tôi trong Bảng 4 cho thấy, bất chấp việc thiếu tinh chỉnh đặc thù cho tác vụ, mô hình của chúng tôi hoạt động tốt một cách đáng ngạc nhiên, cho kết quả tốt hơn tất cả mô hình đã báo cáo trước đó, ngoại trừ Recurrent Neural Network Grammar [8].

Trái với các mô hình RNN sequence-to-sequence [37], Transformer vượt qua BerkeleyParser [29] ngay cả khi chỉ huấn luyện trên tập huấn luyện WSJ gồm 40K câu.

7 Kết luận

Trong công trình này, chúng tôi đã trình bày Transformer, mô hình chuyển đổi chuỗi đầu tiên dựa hoàn toàn vào chú ý, thay thế các lớp hồi quy vốn được dùng phổ biến nhất trong kiến trúc encoder-decoder bằng tự chú ý đa đầu.

Đối với các tác vụ dịch, Transformer có thể được huấn luyện nhanh hơn đáng kể so với các kiến trúc dựa trên lớp hồi quy hoặc tích chập. Trên cả hai tác vụ dịch WMT 2014 English-to-German và WMT 2014 English-to-French, chúng tôi đạt mức hiện đại nhất mới. Ở tác vụ đầu, mô hình tốt nhất của chúng tôi vượt trội ngay cả so với tất cả ensemble đã được báo cáo trước đó.

Chúng tôi kỳ vọng vào tương lai của các mô hình dựa trên chú ý và dự định áp dụng chúng cho các tác vụ khác. Chúng tôi dự định mở rộng Transformer sang các bài toán liên quan đến những phương thức đầu vào và đầu ra khác văn bản, đồng thời khảo sát các cơ chế chú ý cục bộ, bị giới hạn, để xử lý hiệu quả các đầu vào và đầu ra lớn như hình ảnh, âm thanh và video. Làm cho quá trình sinh ít tuần tự hơn là một mục tiêu nghiên cứu khác của chúng tôi.

Mã nguồn chúng tôi dùng để huấn luyện và đánh giá các mô hình có tại <https://github.com/tensorflow/tensor2tensor>.

Lời cảm ơn Chúng tôi biết ơn Nal Kalchbrenner và Stephan Gouws vì những nhận xét, chỉnh sửa và nguồn cảm hứng hữu ích của họ.

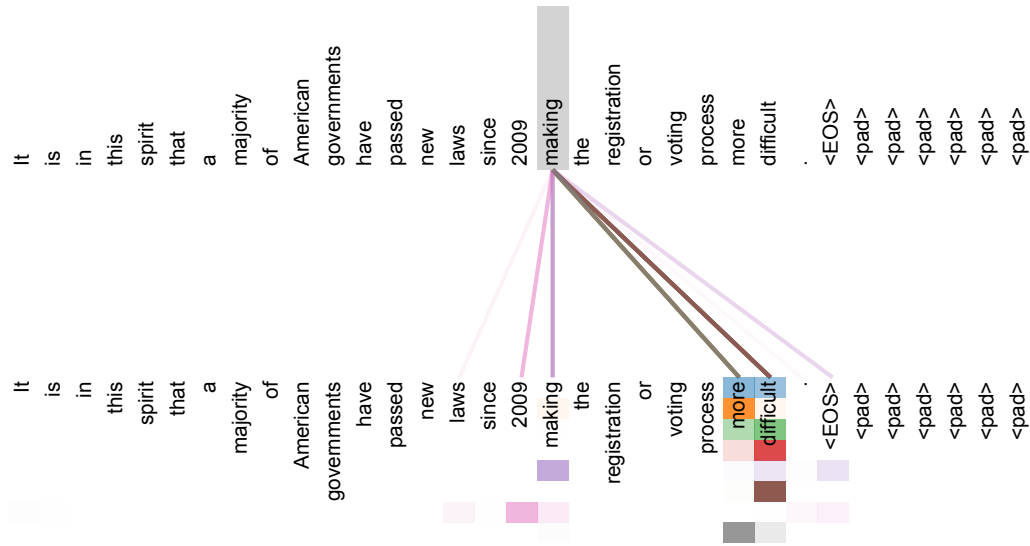
Tài liệu tham khảo

- [1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- [2] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *CoRR*, abs/1409.0473, 2014.
- [3] Denny Britz, Anna Goldie, Minh-Thang Luong, and Quoc V. Le. Massive exploration of neural machine translation architectures. *CoRR*, abs/1703.03906, 2017.
- [4] Jianpeng Cheng, Li Dong, and Mirella Lapata. Long short-term memory-networks for machine reading. *arXiv preprint arXiv:1601.06733*, 2016.

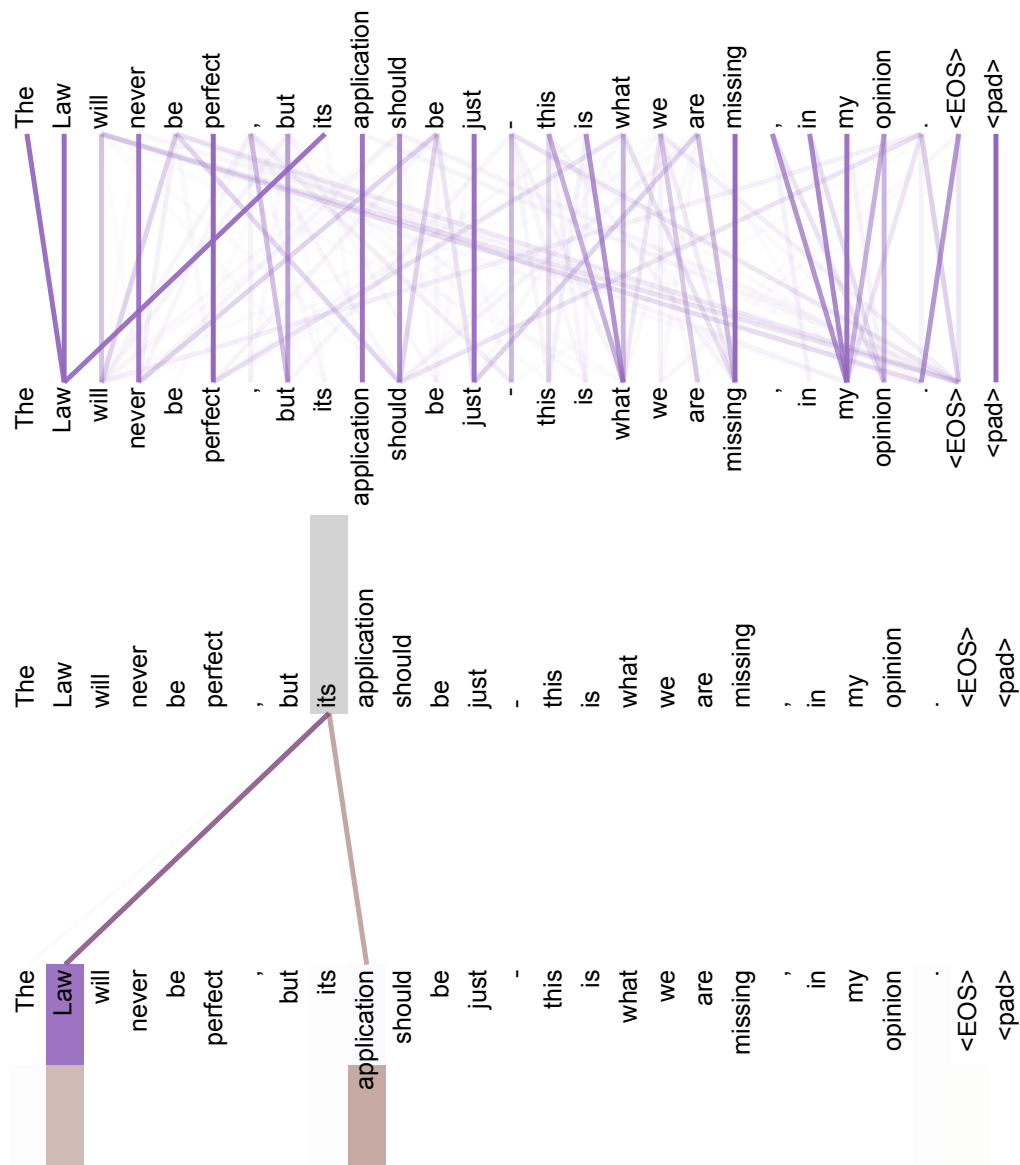
- [5] Kyunghyun Cho, Bart van Merriënboer, Çağlar Gülçehre, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. *CoRR*, abs/1406.1078, 2014.
- [6] Francois Chollet. Xception: Deep learning with depthwise separable convolutions. *arXiv preprint arXiv:1610.02357*, 2016.
- [7] Junyoung Chung, Çağlar Gülçehre, Kyunghyun Cho, and Yoshua Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. *CoRR*, abs/1412.3555, 2014.
- [8] Chris Dyer, Adhiguna Kuncoro, Miguel Ballesteros, and Noah A. Smith. Recurrent neural network grammars. In *Proc. of NAACL*, 2016.
- [9] Jonas Gehring, Michael Auli, David Grangier, Denis Yarats, and Yann N. Dauphin. Convolutional sequence to sequence learning. *arXiv preprint arXiv:1705.03122v2*, 2017.
- [10] Alex Graves. Generating sequences with recurrent neural networks. *arXiv preprint arXiv:1308.0850*, 2013.
- [11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.
- [12] Sepp Hochreiter, Yoshua Bengio, Paolo Frasconi, and Jürgen Schmidhuber. Gradient flow in recurrent nets: the difficulty of learning long-term dependencies, 2001.
- [13] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [14] Zhongqiang Huang and Mary Harper. Self-training PCFG grammars with latent annotations across languages. In *Proceedings of the 2009 Conference on Empirical Methods in Natural Language Processing*, pages 832–841. ACL, August 2009.
- [15] Rafal Jozefowicz, Oriol Vinyals, Mike Schuster, Noam Shazeer, and Yonghui Wu. Exploring the limits of language modeling. *arXiv preprint arXiv:1602.02410*, 2016.
- [16] Łukasz Kaiser and Samy Bengio. Can active memory replace attention? In *Advances in Neural Information Processing Systems, (NIPS)*, 2016.
- [17] Łukasz Kaiser and Ilya Sutskever. Neural GPUs learn algorithms. In *International Conference on Learning Representations (ICLR)*, 2016.
- [18] Nal Kalchbrenner, Lasse Espeholt, Karen Simonyan, Aaron van den Oord, Alex Graves, and Koray Kavukcuoglu. Neural machine translation in linear time. *arXiv preprint arXiv:1610.10099v2*, 2017.
- [19] Yoon Kim, Carl Denton, Luong Hoang, and Alexander M. Rush. Structured attention networks. In *International Conference on Learning Representations*, 2017.
- [20] Diederik Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *ICLR*, 2015.
- [21] Oleksii Kuchaiev and Boris Ginsburg. Factorization tricks for LSTM networks. *arXiv preprint arXiv:1703.10722*, 2017.
- [22] Zhouhan Lin, Minwei Feng, Cicero Nogueira dos Santos, Mo Yu, Bing Xiang, Bowen Zhou, and Yoshua Bengio. A structured self-attentive sentence embedding. *arXiv preprint arXiv:1703.03130*, 2017.
- [23] Minh-Thang Luong, Quoc V. Le, Ilya Sutskever, Oriol Vinyals, and Łukasz Kaiser. Multi-task sequence to sequence learning. *arXiv preprint arXiv:1511.06114*, 2015.
- [24] Minh-Thang Luong, Hieu Pham, and Christopher D Manning. Effective approaches to attention-based neural machine translation. *arXiv preprint arXiv:1508.04025*, 2015.

- [25] Mitchell P Marcus, Mary Ann Marcinkiewicz, and Beatrice Santorini. Building a large annotated corpus of english: The penn treebank. *Computational linguistics*, 19(2):313–330, 1993.
- [26] David McClosky, Eugene Charniak, and Mark Johnson. Effective self-training for parsing. In *Proceedings of the Human Language Technology Conference of the NAACL, Main Conference*, pages 152–159. ACL, June 2006.
- [27] Ankur Parikh, Oscar Täckström, Dipanjan Das, and Jakob Uszkoreit. A decomposable attention model. In *Empirical Methods in Natural Language Processing*, 2016.
- [28] Romain Paulus, Caiming Xiong, and Richard Socher. A deep reinforced model for abstractive summarization. *arXiv preprint arXiv:1705.04304*, 2017.
- [29] Slav Petrov, Leon Barrett, Romain Thibaux, and Dan Klein. Learning accurate, compact, and interpretable tree annotation. In *Proceedings of the 21st International Conference on Computational Linguistics and 44th Annual Meeting of the ACL*, pages 433–440. ACL, July 2006.
- [30] Ofir Press and Lior Wolf. Using the output embedding to improve language models. *arXiv preprint arXiv:1608.05859*, 2016.
- [31] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. *arXiv preprint arXiv:1508.07909*, 2015.
- [32] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.
- [33] Nitish Srivastava, Geoffrey E Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1):1929–1958, 2014.
- [34] Sainbayar Sukhbaatar, Arthur Szlam, Jason Weston, and Rob Fergus. End-to-end memory networks. In C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, and R. Garnett, editors, *Advances in Neural Information Processing Systems 28*, pages 2440–2448. Curran Associates, Inc., 2015.
- [35] Ilya Sutskever, Oriol Vinyals, and Quoc VV Le. Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems*, pages 3104–3112, 2014.
- [36] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. *CoRR*, abs/1512.00567, 2015.
- [37] Vinyals & Kaiser, Koo, Petrov, Sutskever, and Hinton. Grammar as a foreign language. In *Advances in Neural Information Processing Systems*, 2015.
- [38] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. Google’s neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*, 2016.
- [39] Jie Zhou, Ying Cao, Xuguang Wang, Peng Li, and Wei Xu. Deep recurrent models with fast-forward connections for neural machine translation. *CoRR*, abs/1606.04199, 2016.
- [40] Muhua Zhu, Yue Zhang, Wenliang Chen, Min Zhang, and Jingbo Zhu. Fast and accurate shift-reduce constituent parsing. In *Proceedings of the 51st Annual Meeting of the ACL (Volume 1: Long Papers)*, pages 434–443. ACL, August 2013.

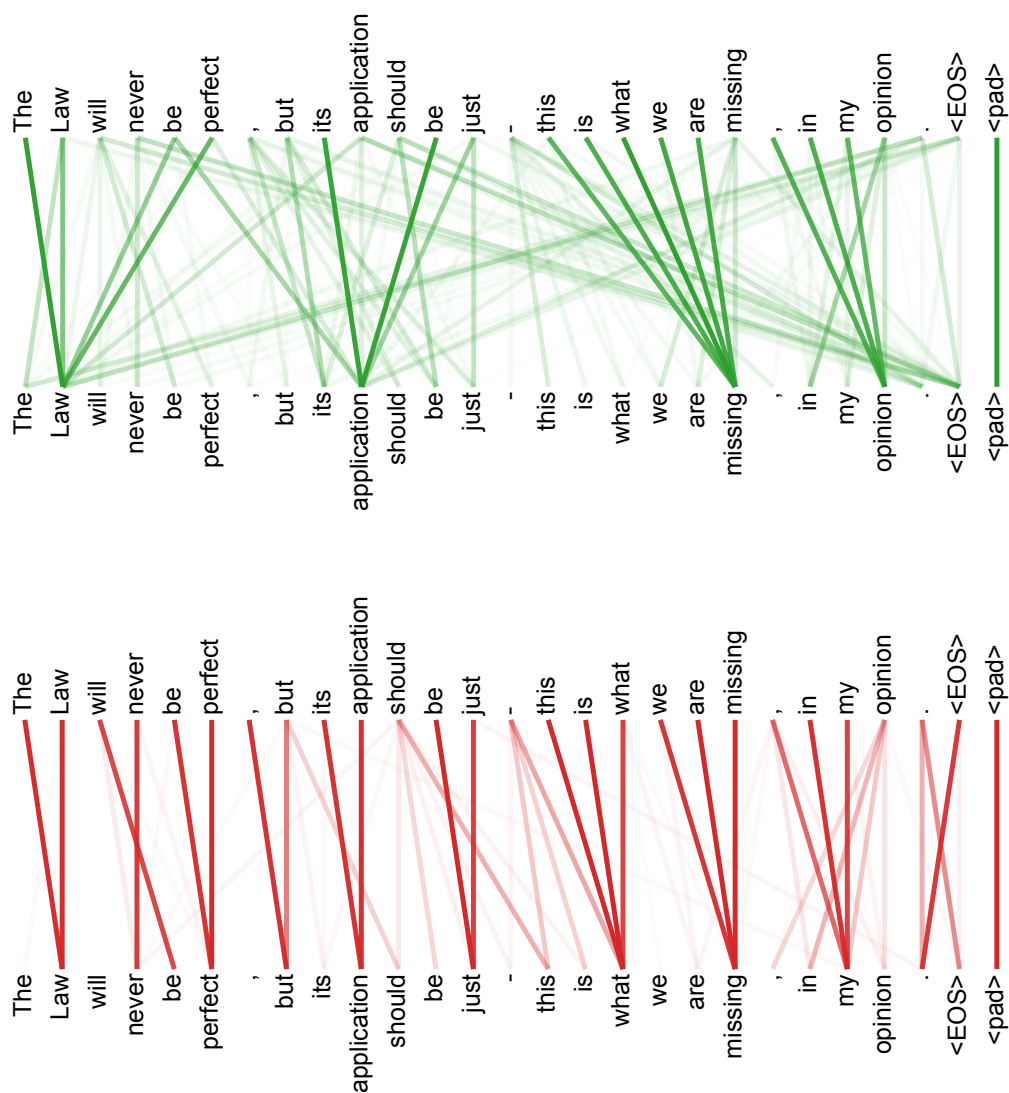
Trực quan hóa chú ý



Hình 3: Một ví dụ về cơ chế chú ý theo dõi các phụ thuộc khoảng cách xa trong tự chú ý của encoder ở lớp 5 trong 6 lớp. Nhiều đầu chú ý tập trung vào một phụ thuộc xa của động từ ‘making’, hoàn thiện cụm ‘making...more difficult’. Ở đây chỉ hiển thị chú ý cho từ ‘making’. Các màu khác nhau biểu thị các đầu khác nhau. Xem tốt nhất ở bản màu.



Hình 4: Hai đầu chú ý, cũng ở lớp 5 trong 6 lớp, dường như liên quan đến giải quyết đồng tham chiếu. Trên: toàn bộ chú ý của đầu 5. Dưới: chú ý được cô lập chỉ từ từ 'its' đối với các đầu chú ý 5 và 6. Lưu ý rằng các chú ý rất sắc nét đối với từ này.



Hình 5: Nhiều đầu chú ý thể hiện hành vi dường như liên quan đến cấu trúc của câu. Chúng tôi đưa ra hai ví dụ như vậy ở trên, từ hai đầu khác nhau của tự chú ý encoder tại lớp 5 trong 6 lớp. Các đầu này rõ ràng đã học để thực hiện các tác vụ khác nhau.